HuBMAP - Hacking the Human Vasculature

535521 Selected Topics in Visual Recognition using Deep Learning

徐葆驊、吳念澤、江怜儀、王妤瑄

Group14

GitHub: `https://github.com/benhsu0828/2026version_HuBMAP_Hacking_the_Human_Vasculature.git`

1 Introduction

This project studies the Kaggle competition *HuBMAP – Hacking the Human Vasculature* [1]. The task is instance segmentation: for each 512×512 histology tile, the model must output separate object masks with confidence scores. The final evaluation follows a mean Average Precision (mAP) style metric over instance masks, so both localization quality and mask quality matter.

The training data is heterogeneous. Dataset 1 contains the cleanest annotations and is used as the main validation and fine-tuning source in our experiments. Dataset 2 contains more weak/noisy annotations and is useful for increasing data coverage, but it can also inject label noise. Dataset 3 provides additional whole-slide image (WSI) regions; in our implementation it is treated mainly as extra WSI context in the dataset metadata, while the main training pipelines focus on Dataset 1 and Dataset 2. This setting makes cross-validation design important: neighboring tiles from the same WSI can be visually similar, so random tile-level splits would overestimate generalization.

2 Methodology and related work

First we survey the top Kaggle solutions and identify key design choices. Here are the main models and techniques that influenced our implementation.

2.1 First-place models

- **RTMDet** [2] is a real-time one-stage detector built from a CSPNeXt backbone and a CSPNeXt-PAFPN neck, trained with a dynamic soft-label assigner. The first place uses the largest variant, RTMDet-X, as its best single model and bolts an FCN mask head on top so the same network can be supervised with masks.
- **YOLOX** [3] is an anchor-free member of the YOLO family with a decoupled detection head and SimOTA label assignment. The first place uses YOLOX-X, again augmented with mask supervision, as one ensemble stream.
- **Mask R-CNN** [4] is the canonical two-stage instance segmentor: it extends Faster R-CNN with RoIAlign and a parallel FCN mask branch. In the first-place ensemble the Mask R-CNN stream uses a **Swin Transformer** [5] backbone, a hierarchical vision transformer with shifted-window self-attention, and its mask head re-predicts the final ensemble masks at input size 1440.
- **Weighted Boxes Fusion (WBF)** [6] is the box-level ensembling step that merges overlapping boxes from all streams by confidence-weighted averaging, rather than discarding them as NMS would.

2.2 Third-place models

- **ViT-Adapter** [7] injects spatial priors into a plain pretrained Vision Transformer through a lightweight adapter, letting a strong ViT backbone perform dense prediction. The third place uses the large variant (ViT-Adapter-L) as its highest-scoring backbone.
- **CBNetV2** [8] is a composite-backbone design that links several (here Swin-based) backbones through composite connections to form a higher-capacity feature extractor without training from scratch.
- **DetectoRS** [9] combines a Recursive Feature Pyramid with Switchable Atrous Convolution. In the third place it is paired with the **Hybrid Task Cascade (HTC)** [10] head, which interleaves box and mask prediction across cascade stages, on **ResNeXt-101** [11] and **ResNet-50** [12] backbones to add CNN-style diversity to the transformer models.

2.3 Our models

- **YOLO26x-Seg** [13] is the segmentation variant of Ultralytics' anchor-free single-stage YOLO detector. We used the largest (`x`) configuration to obtain an architecturally different stream from the Cascade/Mask R-CNN family.

- **InternImage-T** [14] is a CNN-based vision foundation backbone whose core operator is the DCNv3 deformable convolution; `T` denotes the tiny variant. We run the pure-PyTorch `DCNv3_pytorch` operator so the model needs no custom CUDA compilation, which makes it easy to package for Kaggle.
- **Cascade Mask R-CNN** [15, 4] extends Mask R-CNN with a multi-stage cascade of detection heads trained at increasing IoU thresholds, which progressively refines box quality; we attach an FCN mask head and a **Feature Pyramid Network (FPN)** [16] neck for multi-scale fusion.

2.4 First-place direction: WSI-aware heterogeneous ensembles

The first-place Kaggle write-up frames the problem as box-quality-driven instance segmentation: AP is dominated by bounding-box localization, while mask prediction can be delegated to a strong mask head after the boxes are fixed [17]. This explains why the public release is not a single semantic-segmentation model. It is a heterogeneous MMDetection ensemble that includes RTMDet-X, YOLOX-X, and Mask R-CNN streams. The released training repository also marks RTMDet-X as the best single model, and its configs add mask supervision through an extra mask head on top of the detector [17].

Several design choices are especially relevant:

- **WSI-aware validation using two of five spatial folds.** The first place does not use a full k-fold cross-validation ensemble. The write-up states that every model is trained with "two different weights (2 of 5 folds)", and the released `tools/prepare_data.py` ships exactly those two folds. Tiles from the clean Dataset 1 of WSI 1 and WSI 2 are sorted by their vertical tile index `i` and conceptually divided into five contiguous 20% bands. Fold 0 holds out the lowest band (`i` $<$ the 20th percentile) as validation (`dval0i`) and trains on the rest (`dtrain0i`); fold 1 holds out the highest band (`i` $>$ the 80th percentile, `dval1i`/`dtrain1i`). Because each validation set is a contiguous spatial band instead of randomly sampled tiles, neighboring tiles from the same WSI cannot leak between train and validation. The author reports this split as harder but more trustworthy than random splitting: about 0.47 bbox / 0.72 segm mAP@60 under random splitting versus 0.43 / 0.68 under the `i` split.
- **EMA as a stabilizer.** The author highlights Exponential Moving Average (EMA) as the most important technique in the experiments. Multiple EMA copies were evaluated from a single training run, which gave a stable basis for model selection.
- **Mask supervision for better boxes.** RTMDet-X already produced strong boxes, but adding mask supervision and random rotation improved bbox mAP. The mask head was therefore used not only for final masks, but also as an auxiliary signal for localization.
- **Box-first ensemble.** The final ensemble uses WBF over three RTMDet models, one YOLOX-X model with mask supervision, and one Mask R-CNN. All five model types are represented by two different fold weights. The final masks are generated from a Mask R-CNN mask head at input size 1440, and the write-up explicitly notes that no TTA was used.

The role of the two folds is therefore not mainly variance estimation; it is a cheap two-member bagging ensemble. In the released configs each of the five model families is trained once per fold: `r`, `s`, and `sb` are all RTMDet-with-mask variants (three RTMDet streams), `y` is YOLOX-X with mask supervision, and `m` is a Swin-backbone Mask R-CNN. Training each family on both folds yields the ten inference streams $\{\texttt{r}, \texttt{s}, \texttt{m}, \texttt{y}, \texttt{sb}\} \times \{\texttt{0i}, \texttt{1i}\}$ that the ensemble fuses, so every test tile is processed by two independently held-out weight sets of every architecture. This is also why our YOLO extension below adds its own four spatial folds: it follows the same fold-as-bagging philosophy, just with more members.

This project therefore uses the first-place pipeline as the main baseline: reproduce the MMDetection ensemble behavior, keep the bbox-first interpretation of the metric, then test whether a new stream improves or hurts the fused box set.

2.5 Third-place direction: multi-stage training

The third-place write-up focuses on how to properly use noisy annotations [18]. The final system uses five MMDetection-based models: two ViT-Adapter-L models, one CBNetV2-Base model, one DetectoRS ResNeXt-101-32x4d model, and one DetectoRS ResNet-50 model. The author reports using only competition data, with no external image data. The key training idea is multi-stage learning: first pretrain on all noisy Dataset 2 WSIs for a short schedule with light augmentation and high learning rate, then fine-tune on clean Dataset 1 for more epochs with heavier augmentation, higher image resolution, and a lower learning rate.

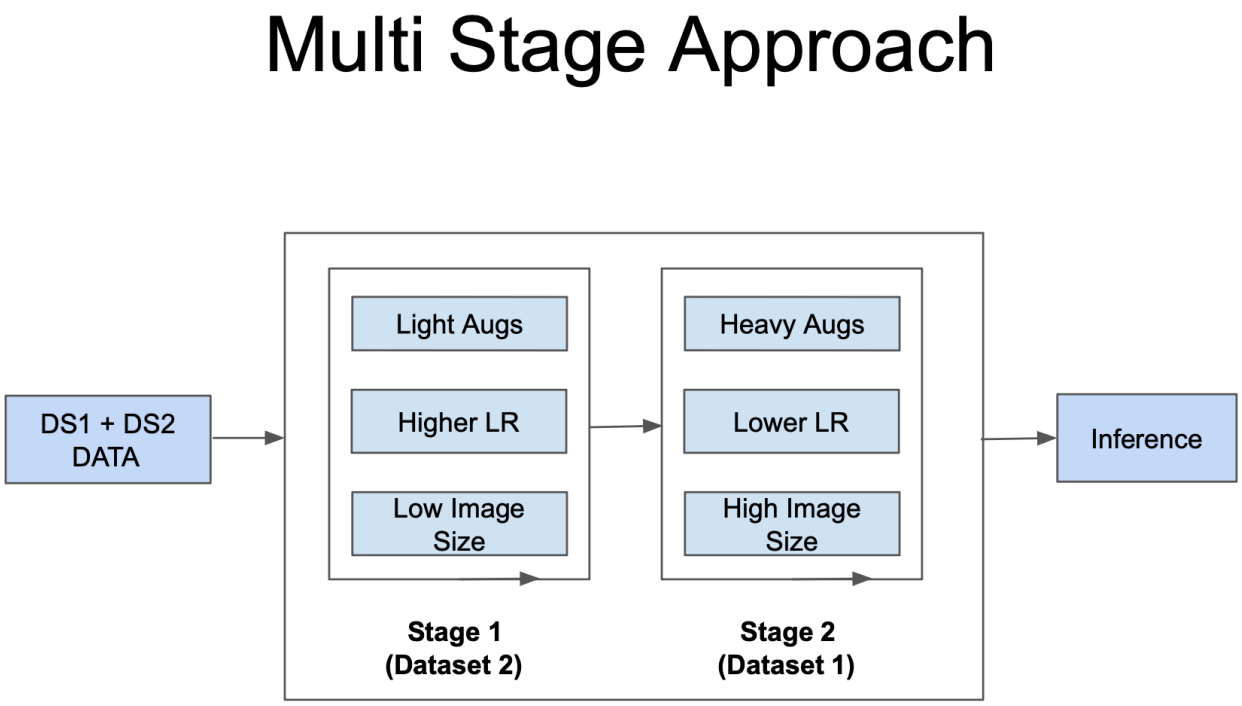

Figure 1: Third-place multi-stage training diagram from the Kaggle write-up [18]. The central idea is to use Dataset 2 for broad noisy pretraining and Dataset 1 for clean high-resolution fine-tuning.

The same write-up also describes a pseudo-label extension for Dataset 3. In that branch, multi-stage models generate predictions, WBF ensembles them, confidence thresholding filters them, and the remaining pseudo labels are combined with Dataset 3 before another multi-stage training pass. The author states that pseudo labels were used in some final ensemble models, although they did not clearly improve leaderboard score. Reported useful techniques include multi-stage training, flip TTA, higher mask-head loss weights for HTC-based models, SGD, WBF, and erosion followed by one dilation step as post-processing.

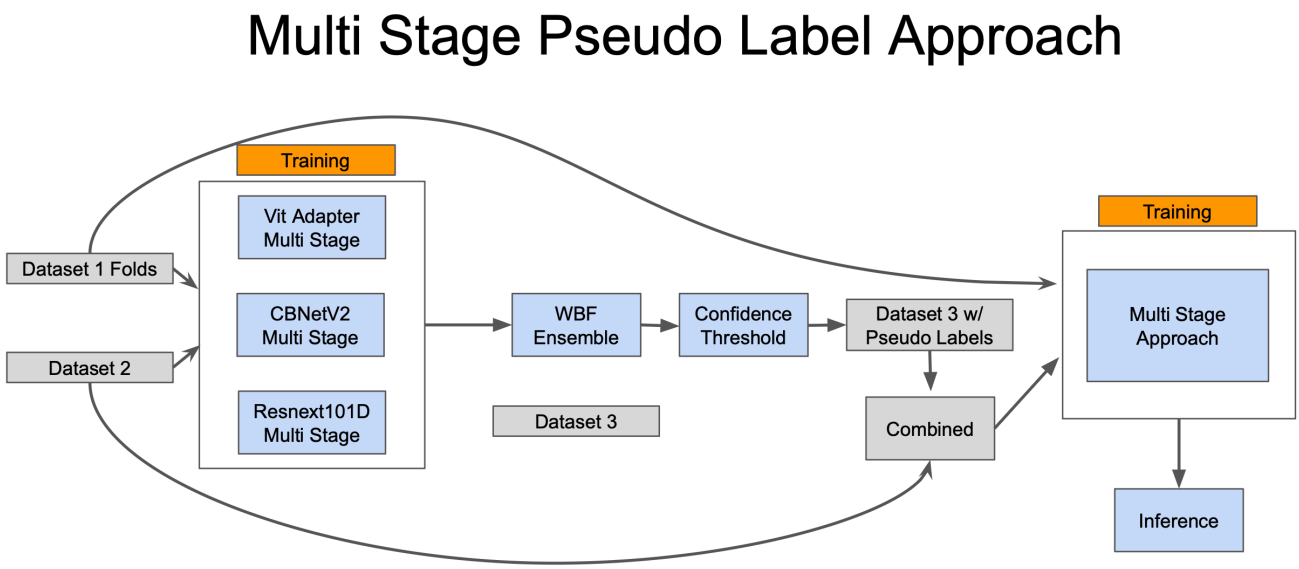

Figure 2: Third-place pseudo-label extension from the Kaggle write-up [18]. Ensemble predictions are thresholded into Dataset 3 pseudo labels and then folded back into another multi-stage training pass.

In our code this appears in `mmdet_v3_solution`: Stage 1 trains on Dataset 2, and Stage 2 fine-tunes on Dataset 1. This strategy is reasonable because Dataset 2 can teach broad morphological priors, while Dataset 1 should correct boundary precision and class noise. However, the original third-place system used much larger high-capacity backbones, higher training and inference resolutions such as 1400–2048 pixels, multi-scale inference, and a five-model ensemble. Our experiment replaces that family with an InternImage-T backbone inside Cascade Mask R-CNN. This makes the experiment informative but not a faithful reproduction of the third-place architecture.

3 Experiment 1: First-place direction — adding a YOLO26-Seg stream

3.1 Motivation

The first experiment asks whether adding a modern YOLO segmentation model can improve the first-place ensemble. The hypothesis is straightforward: YOLO26-Seg is architecturally different from Cascade/Mask R-CNN style models, so it may add useful diversity. If its errors are complementary, WBF should produce better boxes and therefore better final masks.

3.2 Reproduced first-place ensemble baseline

The first-place pipeline is the baseline we extend. Its inference notebook runs ten MMDetection streams:

$$\{\texttt{r0i}, \texttt{r1i}, \texttt{s0i}, \texttt{s1i}, \texttt{m0i}, \texttt{m1i}, \texttt{y0i}, \texttt{y1i}, \texttt{sb0i}, \texttt{sb1i}\}.$$

Each stream outputs bounding boxes, scores, and labels, which are fused by Weighted Boxes Fusion (WBF) [6]. The fused boxes are then passed into a Mask R-CNN style mask branch from `m1i` to produce final instance masks at input size 1440. The original first-place author reported 0.589 private and 0.317 public. Our closest first-place-only rerun used the released first-place models with TTA and scored 0.577 private and 0.316 public, so the reproduction is essentially on par on public but about 0.012 lower on private.

3.3 Data processing

For YOLO experiments we use a dedicated preprocessing pipeline. Each 512×512 tile is padded to 768×768 by stitching neighboring tiles when available, and neighbor polygons are shifted into the padded coordinate frame, which reduces truncation artifacts for vessels crossing tile boundaries. The output is written in Ultralytics YOLO segmentation format with four spatial folds. The split uses Dataset 1 for validation and excludes spatially overlapping Dataset 2 tiles from the corresponding training fold to reduce leakage.

3.4 Method

We trained a four-fold YOLO26x-Seg ensemble using Ultralytics. The training script uses:

- input size 768×768, matching the padded tile size;
- AdamW with fixed learning rate 10^{-3};
- heavy geometric augmentation: 180° rotation, flips, scale, mosaic, mixup, and copy-paste;
- class filtering at inference so that only `blood_vessel` instances are submitted.

The YOLO stream is inserted into the original notebook between the MMDetection inference cells and the WBF cell. Each YOLO fold produces `yolo0.pkl` to `yolo3.pkl`, using the same dictionary format as MMDetection:

$$\texttt{pred\_instances} = \{\texttt{bboxes}, \texttt{scores}, \texttt{labels}\}.$$

This makes YOLO a drop-in WBF stream. In the modified WBF cell, the four YOLO streams are assigned weights $[2, 2, 2, 2]$ and can be enabled with `ABLATION_MODE = baseline`.

3.5 Results and analysis

The YOLO-only validation results were reasonable for a lightweight independent stream, but they did not beat the reproduced first-place ensemble on private leaderboard. The best local fold-level YOLO mask mAP@50:95 ranged from 0.137 to 0.215, as shown in Table 1. The full Kaggle submission records are summarized in Table 2.

Fold	Best epoch	Mask mAP@50	Mask mAP@50:95
0	76	0.226	0.137
1	59	0.255	0.160
2	76	0.305	0.215
3	59	0.302	0.203

Table 1: Best local validation metrics of the YOLO26x-Seg folds from `runs/seg/fold*/results.csv`.

Submission setting	Inference / fusion details	Private	Public
Original first-place author	Public Kaggle write-up score	0.589	0.317
First-place models only	Released first-place models with TTA inference	0.577	0.316
YOLO26-Seg only	Four-fold YOLO26, direct inference, ensemble mask-NMS	0.420	0.387
YOLO26-Seg only	Four-fold YOLO26, 4-way TTA {`orig`, `hflip`, `vflip`, `rot90`}, WBF and weighted mask average	0.515	0.446
YOLO26-Seg + MedSAM-2	YOLO prompts with MedSAM-2 LoRA and vendored MedSAM2	0.258	0.287
First-place + YOLO26-Seg	10 first-place streams plus 4 YOLO streams, WBF weights $[2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2]$	0.572	0.304
First-place + YOLO26-Seg	Same 14-stream WBF with YOLO weights reduced to $[0.5, 0.5, 0.5, 0.5]$ for the last four streams	0.563	0.308
First-place + YOLO26-Seg	Explicit `streams=14` implementation with YOLO weights $[2, 2, 2, 2]$	0.572	0.304

Table 2: Kaggle submission scores from the tested inference variants.

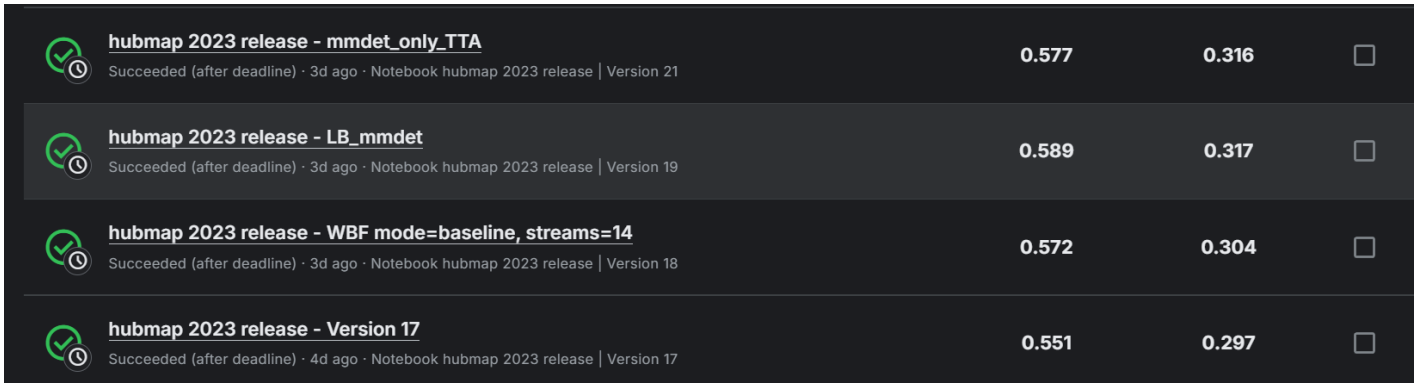

Figure 3: Kaggle submission scores from the tested inference variants.

These records clarify the gap between the experiments and the earlier report draft. YOLO26-Seg improved strongly when moving from direct mask-NMS to 4-way TTA with WBF and weighted mask averaging, rising from 0.420 to 0.515 private. However, adding YOLO streams into the first-place WBF did not improve the first-place system: the best 14-stream result was 0.572 private, below both our first-place-only TTA rerun at 0.577 and the author's 0.589. Lowering YOLO weights to 0.5 improved public score slightly but reduced private score to 0.563. The MedSAM-2 refinement branch also underperformed at 0.258 private, suggesting that prompt-driven mask refinement was not robust to the YOLO box distribution in this setup.

The negative result is still useful. First, YOLO did not provide enough diversity: many mistakes overlapped with the existing detector ensemble, especially on ambiguous vessel-like structures. Second, its bounding boxes were not consistently better than the Cascade/Mask R-CNN family. Since the final pipeline re-predicts masks from fused boxes, box quality dominates the final mAP. Third, the YOLO-only WBF/TTA variant looked attractive on public score (0.446 versus 0.316 for our first-place-only TTA rerun), but it was still weaker on private score (0.515 versus 0.577). Therefore, the public leaderboard alone was not reliable for selecting the final system, and adding a segmentation stream is not sufficient; the stream must improve private-set localization diversity.

4 Experiment 2: Third-place direction — two-stage InternImage Cascade

4.1 Motivation

The second experiment follows the multi-stage idea from high-ranking solutions: first learn from the larger noisy Dataset 2, then fine-tune on the cleaner Dataset 1. Instead of reproducing the original large ViT/CBNet/DetectoRS-style system, we implemented a more practical MMDetection v3 model that can run in our environment and be packaged for Kaggle.

4.2 Data processing

Annotations are converted from `polygons.jsonl` to COCO format. Dataset 1 is split spatially within WSI 1 and WSI 2 to create `dtrain0i.json` (train) and `dval0i.json` (validation), reusing the first-place `i` split, while Dataset 2 is exported as `dtrain_dataset2.json` for the Stage 1 pretraining.

4.3 Method

The model is Cascade Mask R-CNN [15] with:

- **Backbone:** InternImage-T with pure PyTorch DCNv3 operators;
- **Neck:** FPN for multi-scale feature fusion;
- **Head:** 3-stage Cascade RoI head with progressively stricter IoU thresholds;
- **Mask branch:** FCN mask head with mask loss weight increased to 2.0;
- **Resolution:** 1024×1024 training images.

Stage 1 trains on `dtrain_dataset2.json` for 10 epochs with light augmentation and AdamW learning rate 2×10^{-4}. Stage 2 loads the best Stage 1 checkpoint and fine-tunes on `dtrain0i.json` for 20 epochs with stronger augmentation, including random flips, AutoAugment color/geometric policies, and Albumentations elastic transform. The Kaggle inference package uses FP16, six-view TTA, morphological post-processing, and offline MMDetection dependencies.

The third-place write-up specifies a light and a heavy augmentation profile. The light profile (Stage 1) is `RandomFlip` (horizontal/vertical) plus an `AutoAugment` set of Shear, Translate, and Color+Equalize policies, followed by an Albumentations `ShiftScaleRotate`. The heavy profile (Stage 2) keeps these and adds `PhotoMetricDistortion`, `MinIoURandomCrop`, `CutOut`, and an Albumentations block of `RandomRotate90` with a `OneOf` over `ElasticTransform`, `GridDistortion`, and `OpticalDistortion`. Our implementation keeps the same light-then-heavy split but uses a simplified heavy profile — `RandomFlip`, `AutoAugment` (color and geometric), and an Albumentations `RandomRotate90`+`ElasticTransform` — and omits `CutOut`, `MinIoURandomCrop`, `GridDistortion`, and `OpticalDistortion`, which keeps the pipeline light enough for our single-GPU budget.

4.4 Results and analysis

Stage 1 reached local `coco/segm_mAP` 0.130 at epoch 8 on `dval0i`. After fine-tuning on Dataset 1, Stage 2 improved substantially and reached 0.440 local `coco/segm_mAP` at epoch 12. This confirms that the Dataset2 → Dataset1 schedule is useful. However, the result still did not surpass the reproduced first-place ensemble, and it is below what one would expect from a faithful third-place-style system.

Model / stage	Data	Best epoch	BBox mAP	Segm mAP
InternImage Cascade Stage 1	Dataset 2	8	0.181	0.130
InternImage Cascade Stage 2	Dataset 1	12	0.407	0.440

Table 3: Local MMDetection validation metrics on `dval0i`. These are not directly comparable to Kaggle private leaderboard scores.

We also packaged the Stage 2 checkpoint for Kaggle and submitted it with the FP16 / six-view-TTA / morphological pipeline. The submission score depends strongly on the detection score threshold, as shown in Figure 4. Raising `--score-thr` to 0.3 gave 0.372 private and 0.179 public, while lowering it to the code default of 0.001 improved both to 0.395 private and 0.241 public. This matches the design note in `kaggle_infer.py`: because the HuBMAP metric integrates average precision over the confidence ranking, discarding low-confidence detections early throws away recall that the AP integral would otherwise reward. A low threshold keeps the full ranked

list and lets the metric, not a hard cutoff, decide where precision and recall trade off. The single-model InternImage stream therefore peaks at 0.395 private, which is consistent with its local segmentation mAP being far below the ten-stream first-place ensemble.

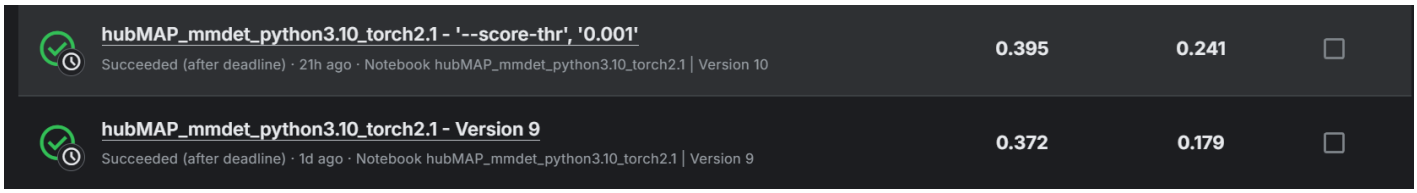

Figure 4: Kaggle leaderboard scores of the two-stage InternImage-T Cascade Mask R-CNN, using FP16, six-view TTA, and morphological post-processing. Lowering the score threshold from 0.3 to the code default 0.001 improves both private and public scores.

For reference, the third-place solution itself scored 0.583 private and 0.619 public [18]. The gap between its public and private scores is tiny, and even slightly reversed, in sharp contrast to the first place, whose public score (0.317) sits far below its private score (0.589). This points to much less distribution shift across the two leaderboard splits: because the third place's later stage still drew on the broad Dataset 2 (and Dataset 3) WSIs, the model generalizes almost identically to public and private tiles. Our two-stage InternImage-T inherits the same schedule but, with a far smaller backbone, a single stream, and lower resolution, peaks at 0.395 / 0.241 — the multi-stage idea transfers, but the capacity does not.

The main reason it stays below the ensemble is architecture mismatch. The referenced multi-stage solution direction relies on very large transformer-style backbones and detector ensembles. Our InternImage-T model is much smaller and is a different system. A second reason is Dataset 2 noise: a compact model may absorb noisy masks more directly, whereas a larger pretrained backbone can be more robust. Finally, our implementation used only a single InternImage Cascade stream. The first-place baseline derives much of its strength from ensembling and from using a dedicated high-resolution mask re-prediction stage.

5 Discussion

The two experiments show that leaderboard-level instance segmentation is a systems problem, not only a model-choice problem. The first-place pipeline works because it separates responsibilities: many models propose boxes, WBF filters and averages localization evidence, and a strong mask branch converts fused boxes into final masks. This structure also explains why the YOLO experiment did not help the final ensemble. YOLO26-Seg produced a reasonable standalone result after WBF/TTA (0.515 private, 0.446 public), but adding it to the first-place streams reduced the private score from our first-place-only 0.577 to 0.572.

The InternImage experiment gives a different lesson. Multi-stage training is useful: Stage 2 improved the local segmentation mAP from 0.130 to 0.440, and on Kaggle the single stream reached 0.395 private / 0.241 public once the score threshold was kept low. But borrowing only the training schedule is not enough to reproduce a top solution. Backbone capacity, resolution, label-noise robustness, and ensemble integration all interact. InternImage-T was attractive because it was easier to package and run, especially with a pure PyTorch DCNv3 implementation, but this practical choice also limited its ceiling.

Overall, our best direction remains the reproduced first-place MMDetection ensemble. The YOLO and InternImage experiments are valuable ablations: YOLO tested whether simple diversity improves WBF, while InternImage tested whether Dataset2 pretraining transfers to Dataset1 fine-tuning. The submission gap is now clear: the original author reported 0.589 private, our first-place-only TTA rerun reached 0.577, and the best YOLO-augmented first-place ensemble reached 0.572. Future improvements should therefore focus on better box proposals, higher-capacity backbones, stronger WSI-aware validation, and more careful fusion rather than simply adding more segmentation outputs.

6 Conclusion

We reproduced the structure of the HuBMAP first-place solution and explored two extensions. The original first-place score was 0.589 private / 0.317 public, while our first-place-only TTA rerun reached 0.577 / 0.316. YOLO26x-Seg alone improved from 0.420 / 0.387 with direct mask-NMS to 0.515 / 0.446 with 4-way TTA, WBF, and weighted mask averaging, but inserting YOLO into the first-place WBF reduced private score to 0.572. The YOLO + MedSAM-2 branch was worse at 0.258 / 0.287. A two-stage Cascade Mask R-CNN with InternImage-T confirmed the benefit of Dataset2 pretraining followed by Dataset1 fine-tuning, scoring 0.395 private / 0.241 public at a low score threshold (versus 0.372 / 0.179 at threshold 0.3), but the smaller backbone and single-model setup remained below the reproduced ensemble baseline. The project therefore supports a central conclusion: for this competition, high-quality box localization and ensemble design are more important than mask sharpness alone.

References

[1] HuBMAP and Kaggle. HuBMAP - hacking the human vasculature. https://www.kaggle.com/c/hubmap-hacking-the-human-vasculature, 2023.

[2] Chengqi Lyu, Wenwei Zhang, Haian Huang, Yue Zhou, Yudong Wang, Yanyi Liu, Shilong Zhang, and Kai Chen. RTMDet: An empirical study of designing real-time object detectors, 2022.

[3] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding YOLO series in 2021, 2021.

[4] Kaiming He, Georgia Gkioxari, Piotr Dollar, and Ross Girshick. Mask R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2961–2969, 2017.

[5] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, 2021.

[6] Roman Solovyev, Weimin Wang, and Tatiana Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models, 2019.

[7] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, and Yu Qiao. Vision transformer adapter for dense predictions. In *International Conference on Learning Representations (ICLR)*, 2023.

[8] Tingting Liang, Xiaojie Chu, Yudong Liu, Yongtao Wang, Zhi Tang, Wei Chu, Jingdong Chen, and Haibin Ling. CBNetV2: A composite backbone network architecture for object detection. *IEEE Transactions on Image Processing*, 31:6893–6906, 2022.

[9] Siyuan Qiao, Liang-Chieh Chen, and Alan Yuille. DetectoRS: Detecting objects with recursive feature pyramid and switchable atrous convolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10213–10224, 2021.

[10] Kai Chen, Jiangmiao Pang, Jiaqi Wang, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. Hybrid task cascade for instance segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4974–4983, 2019.

[11] Saining Xie, Ross Girshick, Piotr Dollar, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1492–1500, 2017.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[13] Ultralytics. Ultralytics YOLO. https://github.com/ultralytics/ultralytics, 2026.

[14] Wenhai Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaowei Hu, Tong Lu, Lewei Lu, Hongsheng Li, Xiaogang Wang, and Yu Qiao. InternImage: Exploring large-scale vision foundation models with deformable convolutions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[15] Zhaowei Cai and Nuno Vasconcelos. Cascade R-CNN: Delving into high quality object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6154–6162, 2018.

[16] Tsung-Yi Lin, Piotr Dollar, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.

[17] tascj. 1st place solution. https://www.kaggle.com/c/hubmap-hacking-the-human-vasculature/discussion/429060, 2023. Kaggle competition write-up.

[18] Nischay Dhankhar. 3rd place solution: How to properly utilise noisy annotations. https://www.kaggle.com/c/hubmap-hacking-the-human-vasculature/discussion/430242, 2023. Kaggle competition write-up.